

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 9 TENTANG BFS DAN DFS



Disusun Oleh :

M.FAJAR FADHILUL ZIKRI

NIM 2311533025

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

SHERLY SUKMADIRA PUTRI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, TAHUN 2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya, sehingga laporan praktikum ini dapat diselesaikan tepat pada waktunya. Laporan ini disusun sebagai dokumentasi dan evaluasi dari materi mata kuliah Struktur Data, yang berfokus pada cara mengorganisasi serta mengelola data secara logis dalam sistem komputer.

Praktikum ini memberikan pemahaman mengenai pentingnya pemilihan metode penyimpanan data yang tepat. Struktur data bukan sekadar cara menyimpan informasi, melainkan fondasi dalam membangun algoritma yang efisien. Dengan pengorganisasian data yang baik, sebuah program dapat memproses informasi dengan kecepatan yang lebih tinggi dan penggunaan memori yang lebih optimal.

Melalui rangkaian praktikum ini, mahasiswa diharapkan tidak hanya menguasai teori pengolahan data, tetapi juga mampu mengimplementasikannya dalam bentuk kode program yang sistematis. Kemampuan untuk menyusun data secara terstruktur merupakan keterampilan krusial yang dibutuhkan untuk memecahkan masalah komputasi yang lebih kompleks di masa mendatang.

Terima kasih kami sampaikan kepada dosen pembimbing atas bimbingan dan ilmu yang diberikan selama proses pembelajaran. Terima kasih juga kepada rekan-rekan mahasiswa yang telah memberikan dukungan serta berbagi ide selama praktikum berlangsung. Penyusun menyadari bahwa laporan ini masih memiliki ruang untuk perbaikan, sehingga kritik dan saran yang membangun akan sangat dihargai.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan membantu dalam memahami betapa pentingnya peran struktur data dalam dunia pengembangan perangkat lunak.

Padang, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB 1 PENDAHULUAN.....	1
1.1 Pengertian Praktikum.....	1
1.2 Tujuan Praktikum.....	1
1.3 Persyaratan Praktikum.....	1
1.4 Tempat dan Waktu Praktikum.....	2
BAB 2 PEMBAHASAN PRAKTIKUM.....	3
2.1 Pengertian Sorting (Insertion, Selection, dan Buble).....	3
2.2 Langkah-Langkah Praktikum.....	3
BAB 3 PENUTUP.....	15
3.1 Kesimpulan.....	15
LAMPIRAN.....	16

BAB 1

PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan cara pengorganisasian, penyimpanan, dan pengelolaan data secara efisien.

Kegiatan ini tidak hanya menekankan pada penguasaan teori mengenai model-model data, tetapi juga pada latihan implementasi struktur penyimpanan yang optimal, analisis efisiensi algoritma, hingga pengujian performa program. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual mengenai tata kelola data dengan kemampuan teknis dalam memecahkan masalah komputasi yang kompleks secara sistematis.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut :

1. Membantu mahasiswa memahami konsep dasar struktur data melalui penerapan langsung.
2. Melatih kemampuan menulis, mengompilasi, dan mengeksekusi program dengan mengikuti aturan sintaksis Struktur data.
3. Meningkatkan keterampilan dalam memecahkan masalah (problem solving) dengan pendekatan algoritmik.
4. Membiasakan mahasiswa bekerja sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
5. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.
6. Mengetahui dan mengaplikasikan Tipe Data stack, queue dll.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.
2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Environment (IDE) yang direkomendasikan.
3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum struktur data mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

BAB 2

PEMBAHASAN PRAKTIKUM

2.1 Pengertian BFS DAN DFS

1. Breadth-First Search (BFS)

BFS adalah algoritma pencarian yang dilakukan secara **melebar**. Algoritma ini mengunjungi node secara bertingkat (layer demi layer), mulai dari node asal (root/start), lalu mengunjungi semua tetangga terdekatnya terlebih dahulu sebelum berpindah ke tingkat berikutnya yang lebih dalam.

Karakteristik Utama BFS:

- **Struktur Data:** Menggunakan antrean **Queue** (prinsip *First-In, First-Out* / FIFO). Node yang pertama kali ditemukan akan menjadi yang pertama dieksplorasi tetangganya.
- **Jalur Terpendek:** BFS menjamin penemuan **jalur terpendek** (*shortest path*) berdasarkan jumlah *edge* (lompatan) pada graf tanpa bobot. Hal ini terbukti pada visualisasi Anda sebelumnya, di mana BFS langsung menemukan jalur paling ringkas lewat area kanan graf.
- **Kelebihan:** Sangat optimal untuk mencari target yang lokasinya dekat dengan titik awal.
- **Kekurangan:** Memakan banyak memori (*space complexity* tinggi) karena harus menyimpan seluruh node pada level tertentu di dalam antrean sebelum pindah ke level berikutnya.

2. Depth-First Search (DFS)

DFS adalah algoritma pencarian yang dilakukan secara **mendalam**. Algoritma ini berjalan menyusuri satu cabang atau jalur tertentu sejauh mungkin hingga mencapai titik buntu (*dead end*), kemudian melakukan **backtracking** (mundur selangkah) untuk mencoba cabang lain yang belum dijelajahi.

Karakteristik Utama DFS:

- **Struktur Data:** Menggunakan struktur data **Stack** (prinsip *Last-In, First-Out* / LIFO) atau memanfaatkan fungsi **Rekursif**.
- **Karakteristik Jalur:** Tidak menjamin jalur terpendek. DFS akan langsung mengembalikan jalur pertama yang ia temukan begitu menyentuh target, meskipun jalur tersebut memutar jauh ke dalam.
- **Kelebihan:** Hemat memori (*space complexity* rendah) karena program hanya perlu menyimpan informasi node di sepanjang jalur aktif yang sedang ditelusuri.
- **Kekurangan:** Berisiko terjebak pada jalur yang sangat dalam atau tak terbatas (*infinite loop*) jika graf memiliki siklus dan tidak dikelola dengan *state* kunjungan yang benar.

Kedua algoritma ini memiliki kompleksitas waktu yang sama, yaitu $O(V + E)$, di mana V adalah jumlah *Vertex* (node) dan E adalah jumlah *Edge* (garis penghubung). Pemilihan metode terbaik selalu bergantung pada kebutuhan spesifik dari struktur graf dan hasil akhir yang ingin dicapai.

2.7 Langkah Langkah Praktikum

Pada praktikum kali ini kita akan membuat 4 buah class yang pertama Adalah **Node_2511533023**:

```
public class Node_2511533023 {
    int data_3023;
    Node_2511533023 left_3023;
    Node_2511533023 right_3023;

    public Node_2511533023(int data_3023) {
        this.data_3023 = data_3023;
        left_3023 = null;
        right_3023 = null;
    }

    public void setLeft_3023(Node_2511533023 node_3023) {
        if (left_3023 == null) {
            left_3023 = node_3023;
        }
    }

    public void setRight_3023(Node_2511533023 node_3023) {
        if (right_3023 == null) {
            right_3023 = node_3023;
        }
    }

    public Node_2511533023 getLeft_3023() {
        return left_3023;
    }

    public Node_2511533023 getRight_3023() {
        return right_3023;
    }

    public int getData_3023() {
        return data_3023;
    }

    public void setData_3023(int data_3023) {
        this.data_3023 = data_3023;
    }

    void printPreorder_3023(Node_2511533023 node_3023) {
        if (node_3023 == null) {
            return;
        }
        System.out.print(node_3023.data_3023 + " ");
        printPreorder_3023(node_3023.left_3023);
        printPreorder_3023(node_3023.right_3023);
    }

    void printPostorder_3023(Node_2511533023 node_3023) {
        if (node_3023 == null) {
            return;
        }
        printPostorder_3023(node_3023.left_3023);
        printPostorder_3023(node_3023.right_3023);
    }
}
```

```

        System.out.print(node_3023.data_3023 + " ");
    }

    void printInorder_3023(Node_2511533023 node_3023) {
        if (node_3023 == null) {
            return;
        }
        printInorder_3023(node_3023.left_3023);
        System.out.print(node_3023.data_3023 + " ");
        printInorder_3023(node_3023.right_3023);
    }

    public String print_3023() {
        return this.print_3023("", true, "");
    }

    public String print_3023(String prefix_3023, boolean isTail_3023, String
sb_3023) {
        if (right_3023 != null) {
            right_3023.print_3023(prefix_3023 + (isTail_3023 ? "| " : " "),
false, sb_3023);
        }

        System.out.println(prefix_3023 + (isTail_3023 ? "\\-- " : "/-- ") +
data_3023);

        if (left_3023 != null) {
            left_3023.print_3023(prefix_3023 + (isTail_3023 ? " " : "| "),
true, sb_3023);
        }

        return sb_3023;
    }
}

```

Kelas `Node_2511533023` berfungsi sebagai cetak biru struktur data *Binary Tree* untuk merepresentasikan hierarki data numerik yang setiap titiknya memiliki maksimal dua cabang anak. Kode ini mengimplementasikan tiga metode penjelajahan mendalam (DFS) yaitu *Preorder*, *Inorder*, dan *Postorder* untuk membaca data secara rekursif, serta dilengkapi fungsi visualisasi karakter teks untuk mencetak bentuk struktur pohon tersebut langsung ke konsol.

Class ke-2 adalah `BTree_2511533023`:

```

public class BTree_2511533023 {
    private Node_2511533023 root_3023;
    private Node_2511533023 currentNode_3023;

    public BTree_2511533023() {
        root_3023 = null;
    }
}

```



```

public boolean search_3023(int data_3023) {
    return search_3023(root_3023, data_3023);
}

private boolean search_3023(Node_2511533023 node_3023, int data_3023) {
    if (node_3023.getData_3023() == data_3023) {
        return true;
    }
    if (node_3023.getLeft_3023() != null) {
        if (search_3023(node_3023.getLeft_3023(), data_3023)) {
            return true;
        }
    }
    if (node_3023.getRight_3023() != null) {
        if (search_3023(node_3023.getRight_3023(), data_3023)) {
            return true;
        }
    }
    return false;
}

public void printInorder_3023() {
    root_3023.printInorder_3023(root_3023);
}

public void printPreOrder_3023() {
    root_3023.printPreorder_3023(root_3023);
}

public void printPostOrder_3023() {
    root_3023.printPostorder_3023(root_3023);
}

public Node_2511533023 getRoot_3023() {
    return root_3023;
}

public boolean isEmpty_3023() {
    return root_3023 == null;
}

public int countNodes_3023() {
    return countNodes_3023(root_3023);
}

private int countNodes_3023(Node_2511533023 node_3023) {
    int count_3023 = 1;
    if (node_3023 == null) {
        return 0;
    } else {
        count_3023 += countNodes_3023(node_3023.getLeft_3023());
        count_3023 += countNodes_3023(node_3023.getRight_3023());
        return count_3023;
    }
}

public void print_3023() {
    root_3023.print_3023();
}

public Node_2511533023 getCurrent() {
    return currentNode_3023;
}

```

```

public void setCurrent(Node_2511533023 node_3023) {
    this.currentNode_3023 = node_3023;
}
public void setRoot(Node_2511533023 root, Node_2511533023 root_3023) {
    this.root_3023 = root_3023;
}
}

```

Kelas BTree_2511533023 merupakan representasi dari pohon biner (*Binary Tree*) yang membungkus objek Node_2511533023 menjadi satu kesatuan struktur data utuh. Kelas ini berfungsi untuk mengelola operasional pohon mulai dari mendefinisikan titik akar (root_3023), menghitung total node, mengecek kekosongan data, hingga memanggil metode penjelajahan (*Inorder*, *Preorder*, *Postorder*, dan pencetakan visual). Selain itu, terdapat fungsi pencarian search_3023 secara rekursif murni yang menyisir seluruh cabang kiri dan kanan untuk mendeteksi keberadaan suatu nilai data di dalam pohon.

Class ke-3 BTreeDriver_2511533023:

```

public class BTreeDriver_2511533023 {
    public static void main(String[] args) {
        //Membuat Pohon
        BTree_2511533023 tree = new BTree_2511533023();
        System.out.print("Jumlah Simpul awal pohon: ");
        System.out.println(tree.countNodes_3023());
        //menambahkan simpul data 1
        Node_2511533023 root = new Node_2511533023(1);
        //menjadikan simpul 1 sebagai root
        tree.setRoot(root, root);
        System.out.println("Jumlah simpul jika hanya ada root");
        System.out.println(tree.countNodes_3023());
        Node_2511533023 node2 = new Node_2511533023(2);
        Node_2511533023 node3 = new Node_2511533023(3);
        Node_2511533023 node4 = new Node_2511533023(4);
        Node_2511533023 node5 = new Node_2511533023(5);
        Node_2511533023 node6 = new Node_2511533023(6);
        Node_2511533023 node7 = new Node_2511533023(7);
        Node_2511533023 node8 = new Node_2511533023(8);
        Node_2511533023 node9 = new Node_2511533023(9);
        root.setLeft_3023(node2);
        node2.setLeft_3023(node4);
        node2.setRight_3023(node5);
        node4.setRight_3023(node8);
        root.setRight_3023(node3);
        node3.setLeft_3023(node6);
        node3.setRight_3023(node7);
        node6.setLeft_3023(node9);
        tree.setCurrent(tree.getRoot_3023());
        System.out.println("menampilkan simpul terakhir: ");
        System.out.println(tree.getCurrent().getData_3023());
        System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
        System.out.println(tree.countNodes_3023());
    }
}

```

```

        System.out.println("inOrder: ");
        tree.printInorder_3023();
        System.out.println("\nPreorder: ");
        tree.printPreOrder_3023();
        System.out.println("\nPostorder: ");
        tree.printPostOrder_3023();
        System.out.println("\nDmenampilkan simpul dalam bentuk pohon");
        tree.print_3023();
    }
}

```

Hasil Output:

```

Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
inOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder:
8 4 5 2 9 6 7 3 1
Dmenampilkan simpul dalam bentuk pohon
|           /-- 7
|      /-- 3
|      |   \-- 6
|      |       \-- 9
|-- 1
|      /-- 5
|-- 2
|      |   /-- 8
|-- 4

```

Kelas BTreeDriver_2511533023 berfungsi sebagai program utama (*main class*) untuk menguji seluruh fungsionalitas dari struktur data *Binary Tree* yang telah dibuat. Di dalam baris kodenya, kelas ini melakukan instansiasi objek pohon, membangun hierarki struktur pohon dengan menghubungkan 9 buah node bertipe data numerik secara manual, lalu melacak dan menampilkan informasi simpul saat ini (*current node*). Terakhir, kelas ini mengeksekusi metode pencetakan hasil penjelajahan data menggunakan tiga algoritma transversal (*Inorder*, *Preorder*, *Postorder*) sekaligus memicu keluaran visual bentuk bagan pohon tersebut secara langsung di konsol terminal.

Class ke-4 GraphTraversal_2511533023:

```
public class GraphTraversal_2511533023 {
    private Map<String, List<String>> graph_3023 = new HashMap<>();

    // Menambahkan edge (graf tak berarah)
    public void addEdge(String node1_3023, String node2_3023) {
        graph_3023.putIfAbsent(node1_3023, new ArrayList<>());
        graph_3023.putIfAbsent(node2_3023, new ArrayList<>());
        graph_3023.get(node1_3023).add(node2_3023);
        graph_3023.get(node2_3023).add(node1_3023);
    }

    // Menampilkan graf awal
    public void printGraph() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node : graph_3023.keySet()) {
            System.out.print(node + " -> ");
            List<String> neighbors_3023 = graph_3023.get(node);
            System.out.println(String.join(", ", neighbors_3023));
        }
        System.out.println();
    }

    // DFS rekursif
    public void dfs(String start_3023) {
        Set<String> visited_3023 = new HashSet<>();
        System.out.println("Penelusuran DFS:");
        dfsHelper(start_3023, visited_3023);
        System.out.println();
    }

    private void dfsHelper(String current_3023, Set<String> visited_3023) {
        if (visited_3023.contains(current_3023)) return;
        visited_3023.add(current_3023);
        System.out.print(current_3023 + " ");
        for (String neighbor_3023 : graph_3023.getOrDefault(current_3023, new
ArrayList<>())) {
            dfsHelper(neighbor_3023, visited_3023);
        }
    }

    // BFS iteratif
    public void bfs(String start_3023) {
        Set<String> visited = new HashSet<>();
        Queue<String> queue = new LinkedList<>();
        queue.add(start_3023);
        visited.add(start_3023);
        System.out.println("Penelusuran BFS:");
        while (!queue.isEmpty()) {
            String current = queue.poll();
            System.out.print(current + " ");
            for (String neighbor_3023 : graph_3023.getOrDefault(current, new
ArrayList<>())) {
                if (!visited.contains(neighbor_3023)) {
                    queue.add(neighbor_3023);
                    visited.add(neighbor_3023);
                }
            }
        }
    }
}
```

```

    }
    System.out.println();
}

// Main
public static void main(String[] args) {
    GraphTraversal_2511533023 graph_3023 = new GraphTraversal_2511533023();

    // Contoh graf: A-B, A-C, B-D, B-E
    graph_3023.addEdge("A", "B");
    graph_3023.addEdge("A", "C");
    graph_3023.addEdge("B", "D");
    graph_3023.addEdge("B", "E");
    // Cetak graf awal
    System.out.println("Graf Awal adalah: ");
    graph_3023.printGraph();
    // Lakukan penelusuran
    graph_3023.dfs("A");
    graph_3023.bfs("A");
}
}

```

Hasil Output:

```

Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E

```

Kelas `GraphTraversal_2511533023` berfungsi sebagai program untuk memodelkan struktur graf tak berarah menggunakan representasi *Adjacency List* berbasis `HashMap` dan `ArrayList`. Program ini mengimplementasikan dua metode penelusuran graf utama, yaitu pencarian mendalam **DFS** secara rekursif serta pencarian melebar **BFS** secara iteratif menggunakan *Queue*. Melalui metode `main`, kode ini menguji keterhubungan lima simpul sampel untuk mendemonstrasikan perbedaan urutan pencetakan node hasil penelusuran kedua algoritma tersebut pada konsol.

BAB 3

PENUTUP

3.1 Kesimpulan

Pada praktikum ini, saya menyimpulkan bahwa pembuatan aplikasi simulator berbasis Java Swing berhasil mempermudah visualisasi proses penelusuran data menggunakan struktur Binary Tree serta penjelajahan graf menggunakan algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS) secara bertahap (step-by-step). Melalui praktikum ini, saya dapat memahami secara langsung bagaimana struktur data non-linear dimanipulasi di dalam memori—baik saat elemen pohon biner dibaca secara rekursif melalui metode Preorder, Inorder, dan Postorder, maupun saat algoritma BFS memanfaatkan antrean Queue untuk menjelajah secara melebar per level dan algoritma DFS memanfaatkan tumpukan Stack untuk menyusuri cabang hingga titik buntu—sekaligus membuktikan bahwa pendekatan visual interaktif jauh lebih efektif dalam memperjelas karakteristik rute terpendek, pelacakan balik (backtracking), dan akumulasi node yang dieksplorasi dibandingkan hanya melihat output teks statis pada konsol.

Tugas Sorting1:

Class PetaKampus_2511533023:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.*;

public class PetaKampus_2511533023 extends JFrame {

    private static final long serialVersionUID = 1L;

    private final Map<String, List<String>> adjList_3023 = new HashMap<>();
    private final Map<String, Point> nodePositions_3023 = new HashMap<>();

    // Status Pencarian
    private List<String> path_3023 = new ArrayList<>();
    private Set<String> visitedNodes_3023 = new LinkedHashSet<>();
    private int exploredCount_3023 = 0;

    // Komponen GUI
    private JComboBox<String> cbStart_3023;
    private JComboBox<String> cbGoal_3023;
    private JPanel panelGraph_3023;
    private JLabel lblJalur_3023;
    private JLabel lblDikunjungi_3023;
    private JLabel lblJumlah_3023;

    public PetaKampus_2511533023() {
        setTitle("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS - NIM: 2511533023");
        setSize(850, 650);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // 1. Inisialisasi Data Peta
        initGraphData_3023();

        // 2. Membuat Panel Atas (Kontrol Input)
```

```

10));
    JPanel panelTop_3023 = new JPanel(new FlowLayout(FlowLayout.LEFT, 15,
10));
    panelTop_3023.setBorder(BorderFactory.createEmptyBorder(5, 10, 5, 10));

    panelTop_3023.add(new JLabel("Lokasi Awal :"));
    String[] lokasiArray_3023 = nodePositions_3023.keySet().toArray(new
String[0]);
    Arrays.sort(lokasiArray_3023);
    cbStart_3023 = new JComboBox<>(lokasiArray_3023);
    panelTop_3023.add(cbStart_3023);

    panelTop_3023.add(new JLabel("Lokasi Tujuan :"));
    cbGoal_3023 = new JComboBox<>(lokasiArray_3023);
    panelTop_3023.add(cbGoal_3023);

    JButton btnBFS_3023 = new JButton("BFS");
    JButton btnDFS_3023 = new JButton("DFS");
    JButton btnReset_3023 = new JButton("RESET");

    // Pewarnaan tombol
    btnBFS_3023.setBackground(new Color(144, 238, 144));
    btnDFS_3023.setBackground(new Color(255, 218, 121));
    btnReset_3023.setBackground(new Color(240, 128, 128));

    panelTop_3023.add(btnBFS_3023);
    panelTop_3023.add(btnDFS_3023);
    panelTop_3023.add(btnReset_3023);
    add(panelTop_3023, BorderLayout.NORTH);

    // 3. Membuat Panel Tengah (Visualisasi Graph)
    panelGraph_3023 = new JPanel() {
        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            displayGraph(g);
        }
    };
    panelGraph_3023.setBackground(Color.WHITE);
    panelGraph_3023.setBorder(BorderFactory.createTitledBorder("VISUALISASI
GRAPH"));
    add(panelGraph_3023, BorderLayout.CENTER);

    // 4. Membuat Panel Bawah (Hasil Pencarian)
    JPanel panelBottom_3023 = new JPanel();
    panelBottom_3023.setLayout(new BoxLayout(panelBottom_3023,
BoxLayout.Y_AXIS));
    panelBottom_3023.setBorder(BorderFactory.createTitledBorder("Hasil
Pencarian"));
    panelBottom_3023.setPreferredSize(new Dimension(800, 130));

    lblJalur_3023 = new JLabel("Jalur : -");
    lblDikunjungi_3023 = new JLabel("Node Dikunjungi : -");
    lblJumlah_3023 = new JLabel("Jumlah Node Dieksplorasi : 0");

    // Mengatur margin tulisan hasil
    lblJalur_3023.setBorder(BorderFactory.createEmptyBorder(2, 10, 2, 10));
    lblDikunjungi_3023.setBorder(BorderFactory.createEmptyBorder(2, 10, 2,
10));
    lblJumlah_3023.setBorder(BorderFactory.createEmptyBorder(2, 10, 5, 10));

```



```

panelBottom_3023.add(lblJalur_3023);
panelBottom_3023.add(lblDikunjungi_3023);
panelBottom_3023.add(lblJumlah_3023);
add(panelBottom_3023, BorderLayout.SOUTH);

// 5. Action Listeners Tombol
btnBFS_3023.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        String start = (String) cbStart_3023.getSelectedItem();
        String goal = (String) cbGoal_3023.getSelectedItem();
        BFS(start, goal);
        displayPath();
    }
});

btnDFS_3023.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        String start = (String) cbStart_3023.getSelectedItem();
        String goal = (String) cbGoal_3023.getSelectedItem();
        DFS(start, goal);
        displayPath();
    }
});

btnReset_3023.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        resetGraph();
    }
});

}

// Menginisialisasi data simpul dan koordinat visualnya
private void initGraphData_3023() {

    nodePositions_3023.put("Gerbang", new Point(50, 250));
    nodePositions_3023.put("Pos Satpam", new Point(180, 250));
    nodePositions_3023.put("Rektorat", new Point(320, 150));
    nodePositions_3023.put("Masjid", new Point(320, 350));
    nodePositions_3023.put("Perpustakaan", new Point(480, 150));
    nodePositions_3023.put("Auditorium", new Point(480, 350));
    nodePositions_3023.put("Fak. Teknik", new Point(620, 100));
    nodePositions_3023.put("Fak. Ekonomi", new Point(620, 250));
    nodePositions_3023.put("Fak. Hukum", new Point(620, 400));
    nodePositions_3023.put("Kantin Pusat", new Point(750, 180));
    nodePositions_3023.put("Asrama", new Point(750, 320));

    // Inisialisasi Adjacency List kosong
    for (String node : nodePositions_3023.keySet()) {
        adjList_3023.put(node, new ArrayList<>());
    }

    // Menambahkan 16 Sisi (Edge) secara bolak-balik (Undirected Graph)
    addEdge_3023("Gerbang", "Pos Satpam");
    addEdge_3023("Pos Satpam", "Rektorat");
    addEdge_3023("Pos Satpam", "Masjid");

```

```

        addEdge_3023("Rektorat", "Perpustakaan");
        addEdge_3023("Rektorat", "Auditorium");
        addEdge_3023("Masjid", "Auditorium");
        addEdge_3023("Perpustakaan", "Fak. Teknik");
        addEdge_3023("Perpustakaan", "Fak. Ekonomi");
        addEdge_3023("Auditorium", "Fak. Ekonomi");
        addEdge_3023("Auditorium", "Fak. Hukum");
        addEdge_3023("Fak. Teknik", "Kantin Pusat");
        addEdge_3023("Fak. Ekonomi", "Kantin Pusat");
        addEdge_3023("Fak. Ekonomi", "Asrama");
        addEdge_3023("Fak. Hukum", "Asrama");
        addEdge_3023("Kantin Pusat", "Asrama");
        addEdge_3023("Gerbang", "Masjid");
    }

    private void addEdge_3023(String source, String dest) {
        adjList_3023.get(source).add(dest);
        adjList_3023.get(dest).add(source);
    }

    // 1. Method BFS()
    public void BFS(String start, String goal) {
        path_3023.clear();
        visitedNodes_3023.clear();
        exploredCount_3023 = 0;

        Queue<String> queue = new LinkedList<>();
        Map<String, String> parentMap = new HashMap<>();

        queue.add(start);
        visitedNodes_3023.add(start);

        boolean found = false;

        while (!queue.isEmpty()) {
            String current = queue.poll();
            exploredCount_3023++;

            if (current.equals(goal)) {
                found = true;
                break;
            }

            for (String neighbor : adjList_3023.get(current)) {
                if (!visitedNodes_3023.contains(neighbor)) {
                    visitedNodes_3023.add(neighbor);
                    parentMap.put(neighbor, current);
                    queue.add(neighbor);
                }
            }
        }

        if (found) {
            String curr = goal;
            while (curr != null) {
                path_3023.add(0, curr);
                curr = parentMap.get(curr);
            }
        }
    }

```

```

    }
}

// 2. Method DFS()
public void DFS(String start, String goal) {
    path_3023.clear();
    visitedNodes_3023.clear();
    exploredCount_3023 = 0;

    Stack<String> stack = new Stack<>();
    Map<String, String> parentMap = new HashMap<>();

    stack.push(start);

    boolean found = false;

    while (!stack.isEmpty()) {
        String current = stack.pop();

        if (!visitedNodes_3023.contains(current)) {
            visitedNodes_3023.add(current);
            exploredCount_3023++;

            if (current.equals(goal)) {
                found = true;
                break;
            }

            // Dibalik agar urutan eksplorasi alfabatis/teratur
            List<String> neighbors = adjList_3023.get(current);
            for (int i = neighbors.size() - 1; i >= 0; i--) {
                String neighbor = neighbors.get(i);
                if (!visitedNodes_3023.contains(neighbor)) {
                    parentMap.put(neighbor, current);
                    stack.push(neighbor);
                }
            }
        }
    }

    if (found) {
        String curr = goal;
        while (curr != null) {
            path_3023.add(0, curr);
            curr = parentMap.get(curr);
        }
    }
}

// 3. Method displayGraph()
public void displayGraph(Graphics g) {
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON);

    // Gambar Garis Sisi (Edge) terlebih dahulu
    g2.setColor(Color.GRAY);
    g2.setStroke(new BasicStroke(2));

```

```

Set<String> drawnEdges = new HashSet<>();

for (String node : adjList_3023.keySet()) {
    Point p1 = nodePositions_3023.get(node);
    for (String neighbor : adjList_3023.get(node)) {
        String edgeKey = node.compareTo(neighbor) < 0 ? node + "-" +
neighbor : neighbor + "-" + node;
        if (!drawnEdges.contains(edgeKey)) {
            Point p2 = nodePositions_3023.get(neighbor);

            if (path_3023.contains(node) && path_3023.contains(neighbor)
&&
                Math.abs(path_3023.indexOf(node) -
path_3023.indexOf(neighbor)) == 1) {
                g2.setColor(Color.RED);
                g2.setStroke(new BasicStroke(4));
                g2.drawLine(p1.x, p1.y, p2.x, p2.y);
                g2.setColor(Color.GRAY);
                g2.setStroke(new BasicStroke(2));
            } else {
                g2.drawLine(p1.x, p1.y, p2.x, p2.y);
            }
            drawnEdges.add(edgeKey);
        }
    }
}

// Gambar Bulatan Simpul (Vertex Node)
int radius = 24;
for (Map.Entry<String, Point> entry : nodePositions_3023.entrySet()) {
    String nodeName = entry.getKey();
    Point p = entry.getValue();

    // Logika Pewarnaan Node
    if (path_3023.contains(nodeName)) {
        g2.setColor(new Color(255, 102, 102));
    } else if (visitedNodes_3023.contains(nodeName)) {
        g2.setColor(new Color(255, 255, 153));
    } else {
        g2.setColor(new Color(173, 216, 230));
    }

    g2.fillOval(p.x - radius, p.y - radius, radius * 2, radius * 2);
    g2.setColor(Color.BLACK);
    g2.setStroke(new BasicStroke(2));
    g2.drawOval(p.x - radius, p.y - radius, radius * 2, radius * 2);

    // Cetak Text Label Nama Node
    g2.setFont(new Font("Arial", Font.BOLD, 12));
    FontMetrics fm = g2.getFontMetrics();
    int textWidth = fm.stringWidth(nodeName);
    g2.drawString(nodeName, p.x - textWidth / 2, p.y + 5);
}

// 4. Method displayPath()
public void displayPath() {
    if (path_3023.isEmpty()) {

```

```

        lblJalur_3023.setText("Jalur : JALUR TIDAK DITEMUKAN!");
    } else {
        lblJalur_3023.setText("Jalur : " + String.join(" -> ", path_3023));
    }

    lblDikunjungi_3023.setText("Node Dikunjungi : " +
visitedNodes_3023.toString());
    lblJumlah_3023.setText("Jumlah Node Dieksplorasi : " +
exploredCount_3023);

    // Meminta panel menggambar ulang visual graph setelah data berubah
    panelGraph_3023.repaint();
}

// 5. Method resetGraph()
public void resetGraph() {
    path_3023.clear();
    visitedNodes_3023.clear();
    exploredCount_3023 = 0;

    lblJalur_3023.setText("Jalur : -");
    lblDikunjungi_3023.setText("Node Dikunjungi : -");
    lblJumlah_3023.setText("Jumlah Node Dieksplorasi : 0");

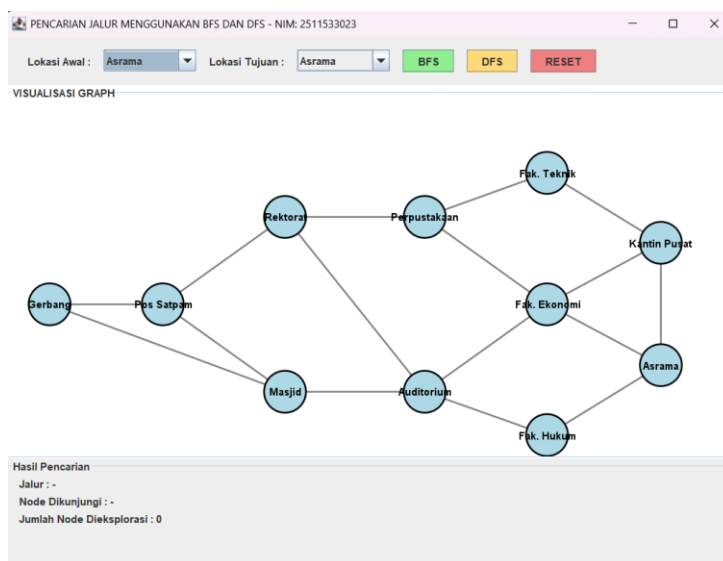
    cbStart_3023.setSelectedIndex(0);
    cbGoal_3023.setSelectedIndex(0);

    panelGraph_3023.repaint();
}

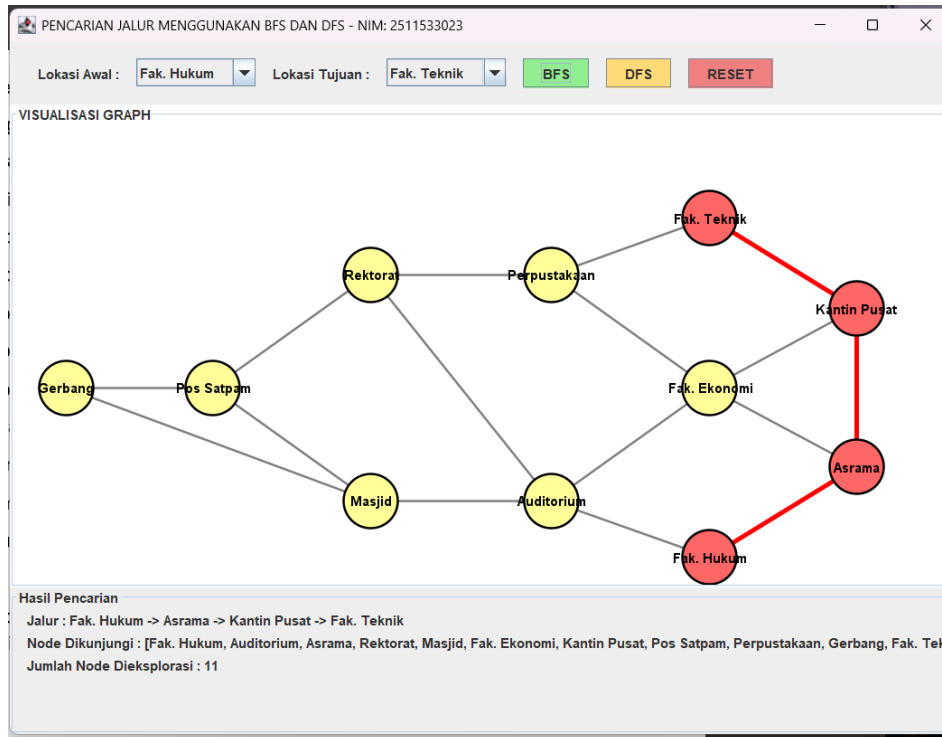
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new PetaKampus_2511533023().setVisible(true);
    });
}
}

```

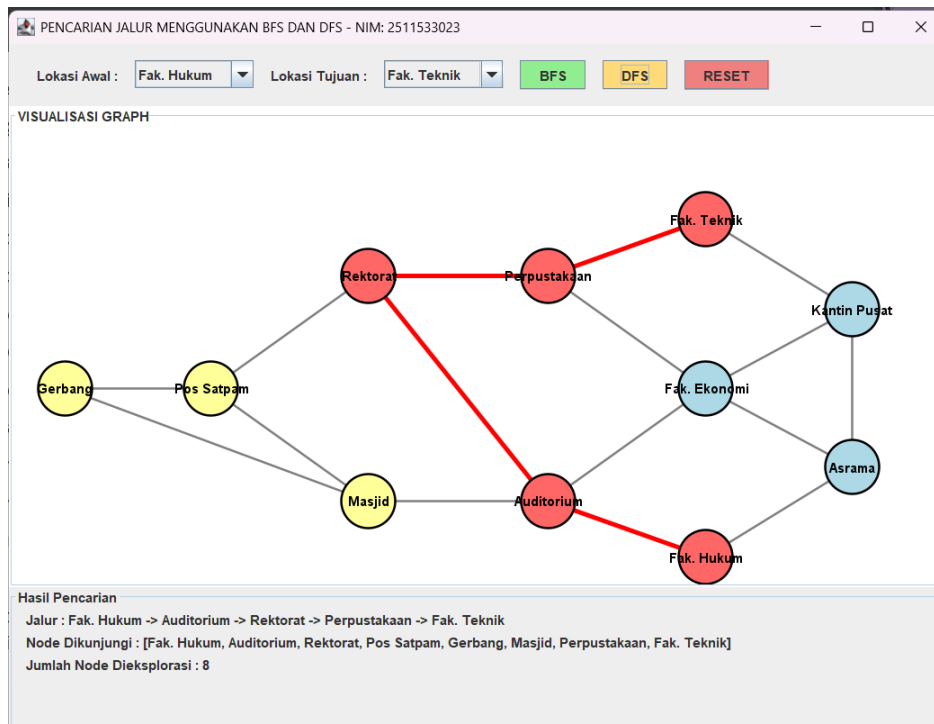
Hasil Output:



BFS:



DFS:



Kelas PetaKampus_2511533023 merupakan aplikasi simulator berbasis Java Swing yang berfungsi untuk memvisualisasikan penelusuran rute terpendek pada graf tak berarah berkepadatan 11 node dan 16 sisi yang merepresentasikan peta lokasi kampus. Program ini menyediakan antarmuka interaktif bagi pengguna untuk memilih lokasi awal serta tujuan, lalu mensimulasikan pencarian jalur menggunakan algoritma *Breadth-First Search* (BFS) dan *Depth-First Search* (DFS) secara langsung. Melalui visualisasi graf dinamis, aplikasi ini secara otomatis mewarnai node serta jalur yang dilewati, menghitung total node yang dieksplorasi, dan mencetak urutan rute hasil pelacakan pada panel informasi bawah sistem.

Kesimpulan perbandingan BFS dan DFS:

Pada praktikum ini, saya menyimpulkan bahwa pembuatan aplikasi simulator berbasis Java Swing berhasil mempermudah visualisasi proses penjelajahan graf tak berarah pada peta kampus menggunakan algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS) secara bertahap (*step-by-step*). Melalui praktikum ini, saya dapat memahami secara langsung bagaimana struktur data non-linear dimanipulasi di dalam memori—baik saat algoritma BFS memanfaatkan antrean *Queue* untuk menjelajah secara melebar per layer demi menjamin penemuan rute terpendek dengan jumlah lompatan (*edge*) minimum, maupun saat algoritma DFS memanfaatkan tumpukan *Stack* untuk melakukan pencarian mendalam menyusuri satu cabang hingga buntu lalu melakukan pelacakan balik (*backtracking*)—sekaligus membuktikan bahwa pendekatan visual interaktif jauh lebih efektif dalam memperjelas perbandingan karakteristik rute, pewarnaan status node, dan akumulasi jumlah node yang dieksplorasi dibandingkan hanya melihat *output* teks statis pada konsol.